Adaptive LLM Model Routing Using Contextual Bandits with Feedback

Rishi Vel. A
Department of M.Tech Computer Science and Engineering (5 Years Integrated)
Sri Sairam Engineering College
Sai Leo Nagar, West Tambaram-44
Chennai, Tamil Nadu, India
sec22cj027@sairamtap.edu.in

Mrs. Kayalvizhi C
Assistant Professor
Department of M.Tech Computer Science and Engineering (5 Years Integrated)
Sri Sairam Engineering College
Sai Leo Nagar, West Tambaram-44
Chennai, Tamil Nadu, India

***Abstract*—Applications that can call several large language models (LLMs) must decide which model should answer each query. Larger models tend to produce stronger answers at higher cost, whereas smaller models are cheaper but may fail on complex tasks. This paper formulates per-query model selection as a contextual-bandit problem and solves it with the Linear Upper Confidence Bound (LinUCB) algorithm. Each query is converted into a compact context vector; LinUCB scores every candidate model by combining an estimated reward with an uncertainty bonus, selects one model, and updates that model's statistics from the observed outcome. The reward combines normalized response quality, cost, and latency with explicit weights. The router is ordinary software and does not modify LLM weights. A pilot study routes 300 GSM8K math word problems (90 development, 210 held-out) among Qwen3-4B, Qwen3-8B, and Qwen3-14B, served in FP8 with vLLM on one RTX 4090, and compares LinUCB with small-only, large-only, random, and rule-based routing by full-information replay over five seeds. On the held-out split the three models reach 94.8%, 95.7%, and 95.2% accuracy, while cost per request rises from 1.42 to 4.71 units. LinUCB learns to send 98.2% of queries to the 4B model, matching its accuracy (94.8%) at 69% lower cost than large-only routing and a cumulative reward of 175.6 versus 139.9; it does not, however, beat the small-only and rule-based baselines (176.0), which are already near-optimal on this single-category workload.**

Keywords*—*Large language models, model routing, contextual bandits, LinUCB, exploration-exploitation, cost-aware inference, feedback-driven learning, LLM serving.

I. Introduction

Large language models are now deployed behind many applications, and an application frequently has access to more than one model. These models differ in response quality, inference latency, and computational or API cost. Sending every request to the largest model can produce strong answers but increases cost and delay for queries that a smaller model could answer; sending every request to the smallest model is cheaper but may fail on multi-step reasoning, code generation, or other complex tasks.

Prior work trades cost against quality by choosing among models. LLM cascades query models in sequence and stop when an answer is judged acceptable [1], while learned routers predict which model a query should be sent to [2]-[5], and RouterBench formalizes their evaluation [6]. Most of these routers are trained offline and remain fixed after deployment. Contextual bandits instead learn from operation: a learner observes a context, chooses one action, and receives a reward only for that action [7]. LinUCB models expected reward as a linear function of the context and adds an upper-confidence exploration bonus [8], [9]. In LLM routing the context describes the query, the actions are candidate models, and the reward expresses the usefulness of the response relative to its cost and latency [10], [11].

This paper makes three contributions. It defines a routing loop that combines interpretable query features, LinUCB, and an explicit quality-cost-latency reward; it specifies a same-family model pool and a replay protocol that compares LinUCB with small-only, large-only, random, and rule-based routing; and it reports a GSM8K pilot with Qwen3-4B, 8B, and 14B on one RTX 4090, including where LinUCB does not help. The router never changes model weights; it only learns which model to select for which kinds of queries. The study asks how to describe a query for routing (RQ1), whether LinUCB beats simple routing under the same reward (RQ2), how the reward weights move the result (RQ3), how quickly routing adapts to change (RQ4), and in which task categories it helps (RQ5).

II. Related Work

FrugalGPT [1] cuts LLM API cost with a cascade that stops once an answer is judged reliable, and Hybrid LLM [2] routes between a small and a large model by predicted difficulty. RouteLLM [3] learns a router from preference data, Shnitzer et al. [4] learn selection rules from benchmarks, Zooter [5] distills reward-model scores into a router, and LLM-Blender [12] combines the outputs of several models at the price of running all of them. These routers are trained offline and then fixed. On the bandit side, UCB methods pick the action with the most optimistic estimate [13]; LinUCB [8] adds context, with analyses by Chu et al. [9] and Abbasi-Yadkori et al. [14]. Logged data allow offline replay evaluation [15], and sliding-window UCB handles changing rewards [16]. MixLLM [10] and PILOT [11] apply contextual bandits to LLM routing with learned embeddings and budget control. Reward quality depends on scoring: references and unit tests are reliable where they apply [17]-[19], while LLM judges show position and verbosity biases [20]. This work keeps the router simple and inspectable and reports measured results against simple baselines, including where it does not win.

III. Problem Formulation and Study Design

A. Contextual-Bandit Formulation

Queries arrive one at a time. For each query a feature extractor produces a short list of numbers that describes it, the router chooses one model from the candidate set, and the system then observes a reward for that model only; the

outcome the other models would have produced remains unknown. This is the contextual-bandit setting [7]: the query description is the context, each model is an action, and the reward scores the usefulness of the answer against its cost and delay.

The router's goal is to collect as much total reward as possible over the whole query stream, which is called the cumulative reward. A second, equivalent view measures what is lost compared with an ideal oracle that always picks the model with the highest reward for each individual query. The total of these per-query losses is the regret. The oracle is not a real policy, since it needs every model's answer in advance, but it can be computed in this study because every model is run on every query offline, and it shows how much room for improvement is left.

B. Candidate Model Pool

The pool contains three Qwen3 models of different sizes [21]: Qwen3-4B as the small model, Qwen3-8B as the medium model, and Qwen3-14B as the large model, loaded from the official checkpoints Qwen/Qwen3-4B-FP8, Qwen/Qwen3-8B-FP8, and Qwen/Qwen3-14B-FP8. One family keeps tokenizer, prompt format, and training recipe fixed, so differences are driven mainly by scale. All three are served with vLLM [22] through one OpenAI-compatible interface on a single NVIDIA RTX 4090 (24 GB). Because bf16 Qwen3-14B does not fit in 24 GB, Qwen's official FP8 checkpoints are used for all three models so precision is identical across the pool. Decoding is greedy (temperature 0, seed 0) with at most 1,024 output tokens, a 120 s timeout, and thinking mode disabled [21]. The models are served one at a time. Cost is a compute proxy: the per-1k-token rate (4.0, 8.0, and 14.0 units per 1,000 tokens for the three sizes) times the request's input plus output tokens, so larger models cost more for the same answer and longer answers cost more on any model.

C. Task Categories and Dataset

The full design covers seven categories with a quality signal for each: general question answering (exact match, e.g., MMLU [17]), reasoning and multi-step problems (final-answer match, e.g., GSM8K [18]), coding (unit tests, e.g., HumanEval [19]), and summarization, writing, information extraction, and instruction following (rubric scores from a validated judge, field-level match, or constraint checks). The pilot reported here covers the reasoning category only: 300 problems sampled with seed 0 from the official GSM8K test split [18], divided into 90 development and 210 held-out test queries. Every problem uses the same prompt: solve the word problem, reason step by step, and give the final answer on the last line in the form '#### <number>'. The numeric scorer takes the number after the last '####' marker, or the last number in the response if the marker is missing, removes thousands separators, and accepts it when it matches the reference up to rounding. Quality is 1 for a match and 0 otherwise; applied to the reference solutions themselves, the scorer reproduces all 1,319 GSM8K test answers. Every query is run on every model (900 outcomes), and each outcome logs the response, latency, input and output tokens, cost, failure status, and quality.

IV. Proposed Routing Framework

Fig. 1 shows the loop: the query is converted into features, LinUCB scores each candidate, one model generates the response, the response is evaluated, a reward is computed, and the chosen model's statistics are updated before the next query.

A. Query Features

Each query is described by 12 numbers, all computed before any model is called. The first measures prompt length on a logarithmic scale between 0 and 1, using 1.3 tokens per word as an estimate. Seven numbers mark which of the seven task categories the query belongs to. Two yes-or-no cues are found by pattern matching: a code cue (code blocks, function or class definitions, imports, error traces) and a reasoning cue (phrases such as 'step by step', 'solve', 'how many', 'prove', or 'explain'). A difficulty value gives, for the query's category, the share of development questions the smallest model got wrong. A constant entry completes the list. A compressed sentence embedding [23] is an optional extension, and the value of each feature group is measured by ablation (E4).

B. LinUCB Scoring and Update

LinUCB [8] keeps a separate, small statistical model for each candidate LLM. For a given model it stores a summary of the feature vectors of the queries that model has answered and of the rewards those answers earned; from this summary it fits a linear estimate that predicts the reward of the model for a new query. Each model's summary starts empty, with a regularization setting ($\lambda = 1$) that keeps early estimates from overreacting to the first few observations.

To choose a model, LinUCB computes a score for every candidate. The score is the predicted reward plus an exploration bonus. The bonus is large when the model has seen few queries similar to the current one and shrinks as such queries accumulate, so the router is optimistic about models it knows little about and gives them an occasional chance. A single coefficient α sets the size of the bonus: $\alpha = 0$ means pure exploitation of current estimates, and larger values mean more exploration. The model with the highest score is called. Once the reward is known, only that model's summary is updated with the query's features and the reward; the other models are left unchanged, because nothing was learned about them.

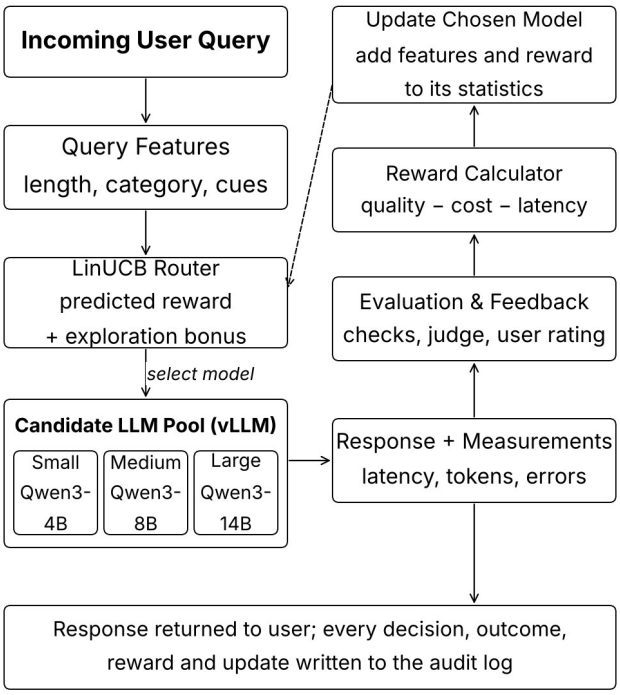


Fig. 1. Proposed adaptive LLM routing loop with contextual-bandit feedback (dashed: the updated statistics drive future routing decisions).

C. Reward Function

The reward turns quality, cost, and latency into one number. It starts from the quality of the answer, which lies between 0 and 1, and subtracts a cost penalty and a latency penalty. Each penalty is the request's cost or latency rescaled to the range 0 to 1, using the smallest and largest values seen on the development split, and then multiplied by a weight that states how much the application cares about it. The pilot weights quality by 1.0, cost by 0.3, and latency by 0.2; the cost range is 0.93 to 6.25 units and the latency range 2.7 to 8.9 s. A perfect, fast, and cheap answer therefore earns close to 1, and a correct but slow and expensive one earns noticeably less.

Failures and timeouts are scored as zero quality while keeping their measured cost and latency, so a cheap but unusable response cannot earn a high reward. Feedback can also come from user ratings or a judge model [20]; judge scores are checked against manual inspection before use, and observations without a trusted quality signal are left out of the update.

V. Experimental Setup

LinUCB is compared with four non-learning policies that share its pool, prompts, and reward. Small-only always calls Qwen3-4B and large-only always calls Qwen3-14B; together they bracket the cheapest and the most capable fixed choice. Random routing picks one of the three models uniformly with a seeded generator and shows what an uninformed mix costs. The rule-based policy maps each task category to the model with the highest mean development reward, which is the strongest simple policy a practitioner could write by hand. None of these policies updates from feedback; only LinUCB learns from the rewards it observes.

Because every query is run on every model, each policy is replayed on the same query stream and sees only the logged outcome of the model it selects; this also makes the per-query oracle computable [15]. Learning policies are first warmed up on the 90 development queries (unscored) and then evaluated on the 210 held-out queries. Each policy is run with five seeds (0-4) that change the query order and the random choices, and results are reported as mean ± standard deviation across seeds [24]. The main comparison uses $\alpha = 1.0$ and $\lambda = 1.0$. Metrics are mean quality, mean and p95 latency, cost per request, failure rate, cumulative reward, regret, and the model-selection distribution. The sweeps vary one factor at a time with $\alpha = 0.25$ unless stated: reward weights (E2), feature groups (E4), the exploration coefficient α (E6), and three bandit variants (linear Thompson sampling, ε-greedy ridge regression with $\varepsilon = 0.1$, and sliding-window LinUCB with a window of 200). Workload shift (E5) and multi-category analysis (E3) require more than one task category and are left to the full study.

One script runs the pilot: it serves each model in turn with vLLM, sends all 300 problems through a resumable collector, frees the GPU for the next model, and finally runs the replay with seeds 0-4.

VI. Results and Analysis

A. Per-Model Outcomes

Table I summarizes the 900 logged outcomes (300 problems × 3 models); latency, output tokens, and cost are held-out means. No request failed or timed out, so the failure rate is 0 for every policy. The three models are close in accuracy: Qwen3-8B is highest at 95.7%, Qwen3-14B reaches 95.2%, and Qwen3-4B 94.8%, a gap of one to two problems out of 210. Cost per request, by contrast, is 2.2 and 3.3 times higher for the 8B and 14B models. Latency does not follow size: the 14B model produced the shortest answers (231 output tokens on average) and had the lowest mean latency, so on this hardware and FP8 setup latency is governed more by answer length than by parameter count. The development split is harder for the small model (86.7% versus 92.2% and 94.4%), a reminder that 90 queries give noisy estimates. On the test split at least one model is correct on 209 of 210 problems; Qwen3-4B is wrong but a larger model is right on 10 of them.

The errors overlap little. Of the 11, 9, and 10 test problems missed by the 4B, 8B, and 14B models, only one is missed by all three, and 5, 7, and 5 are missed by one model alone. Wrong answers are longer than right ones for every model (355 versus 245 output tokens for Qwen3-4B and 456 versus 272 for Qwen3-8B), and only one response, from the 4B model, reached the 1,024-token limit. Decoding speed is nearly constant across sizes at 21 to 23 ms per output token on the FP8 checkpoints, which explains why latency tracks answer length. A router could exploit these patterns only with features that predict an individual model's failure before generation; the length, category, and cue features used here do not, which the ablation below confirms.

TABLE I
PER-MODEL OUTCOMES ON GSM8K AND LINUCB SELECTION SHARE

Model	Acc. Dev / Test	Latency (s)	Output Tokens	Cost / Req.	LinUCB Share
Qwen3-4B	0.867 / 0.948	5.37	251	1.42	0.982
Qwen3-8B	0.922 / 0.957	5.83	280	3.08	0.012
Qwen3-14B	0.944 / 0.952	5.24	231	4.71	0.006

B. Overall Comparison (E1)

TABLE II
POLICY COMPARISON ON THE HELD-OUT SPLIT (MEAN ± SD OVER 5 SEEDS)

Policy	Accuracy	Latency Mean / p95 (s)	Cost / Req.	Cum. Reward	Regret
Small-only	0.948	5.37 / 9.29	1.42	176.0	8.9
Large-only	0.952	5.24 / 8.30	4.71	139.9	45.0
Random	0.946 ± 0.009	5.46 / 8.52	3.13 ± 0.10	155.2 ± 2.9	29.7 ± 2.9
Rule-based	0.948	5.37 / 9.29	1.42	176.0	8.9
LinUCB	0.948	5.36 / 9.27	1.46 ± 0.01	175.6 ± 0.1	9.3 ± 0.1

Table II and Fig. 2 report the main comparison. Under the default weights the cost and latency penalties outweigh the small accuracy gains of the larger models, so the 4B model is the best fixed choice. LinUCB learns this: it routes 98.2% of test queries to Qwen3-4B (Table I), reaches the same mean quality (0.948), and costs 1.460 per request, 69% less than large-only routing (4.706) at 0.4 percentage points lower accuracy. Its cumulative reward of 175.6 ± 0.1 is 35.7 higher than large-only and 20.4 higher than random routing. It does not beat small-only or rule-based routing (176.0); its residual exploration of the larger models (1.8% of queries) costs 0.4 reward units, i.e., 0.002 per query. The rule-based policy coincides with small-only because the pilot has a single category whose best development model is Qwen3-4B. Regret against the per-query oracle stays at 8.9 even for the best fixed policy because, with binary quality, the oracle also knows which model happened to be right on each problem. RQ2 is therefore answered negatively for this workload: LinUCB matches but does not exceed the best fixed policy, which is expected when one model dominates and no context separates the queries.

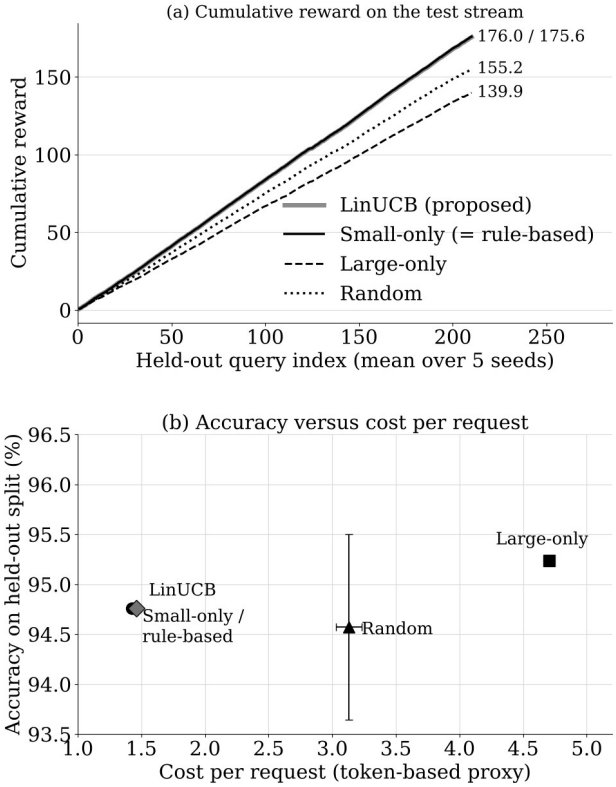


Fig. 2. GSM8K pilot on the held-out split: (a) cumulative reward of each policy over the 210-query stream; (b) accuracy versus cost per request (error bars: SD over 5 seeds).

TABLE III
SWEEPS ON THE HELD-OUT SPLIT (MEAN OVER 5 SEEDS; E2 WEIGHTS ARE COST / LATENCY)

Setting	Reward	Acc.	Cost	Regret	Share 4B/8B/14B
E2: weights 0 / 0	0.948	0.948	1.42	10.0	1.00/0.00/0.00
E2: weights 0.1 / 0.05	0.918	0.948	1.42	9.6	1.00/0.00/0.00
E2: weights 0.3 / 0.2	0.838	0.948	1.42	8.9	1.00/0.00/0.00
E2: weights 0.6 / 0.4	0.687	0.950	1.76	16.4	0.80/0.20/0.00
E2: weights 1.0 / 0.6	0.610	0.948	1.42	6.2	1.00/0.00/0.00
E6: α = 0, 0.1, 0.25	0.838	0.948	1.42	8.9	1.00/0.00/0.00
E6: α = 0.5	0.799	0.951	2.10	17.2	0.60/0.39/0.01
E6: α = 1.0	0.836	0.948	1.46	9.3	0.98/0.01/0.01
E6: α = 2.0	0.813	0.953	1.95	14.2	0.75/0.17/0.07
Linear Thompson sampling	0.823	0.951	1.77	12.0	0.88/0.03/0.09
ε-greedy ridge (ε = 0.1)	0.810	0.949	1.89	14.8	0.76/0.21/0.03
Sliding-window LinUCB	0.838	0.948	1.42	8.9	1.00/0.00/0.00

C. Reward Weights, Exploration, Features, and Variants

Table III reports the sweeps. Reward weights (E2, RQ3): with no cost or latency penalty the best fixed policy is large-only (mean reward 0.952), yet LinUCB still selects only Qwen3-4B and reaches 0.948: initial score ties are broken toward the first model, and once Qwen3-4B has accumulated positive reward during warm-up the bonus at α = 0.25 is too small to try the larger models. For every non-zero weight setting small-only is the best fixed policy, and LinUCB matches it except at cost and latency weights of 0.6 and 0.4, where it shifted 20% of queries to Qwen3-8B and lost 0.042 per query. Exploration (E6): $\alpha \leq 0.25$ is equivalent to always choosing the 4B model; larger α buys at most 0.5 percentage points of accuracy (0.953 at α = 2.0) at 37% higher cost and lower reward, and the effect is not monotone (α = 0.5 is worse than α = 1.0), reflecting seed-to-seed variation on a small test set. Feature ablation (E4, RQ1): removing length, category, cue, or difficulty features, or keeping only the bias term, leaves every metric unchanged (mean reward 0.838), so on a single-category workload the features carry no routing signal. The reason is visible in the inputs: the shared prompt template contains 'solve' and 'step by step', so the reasoning cue is 1 for all 300 prompts; the code cue fires on only 5; the category one-hot is constant; and the difficulty prior is a single value (0.133, from Qwen3-4B's 86.7% development accuracy). Prompt length is the only feature that varies, and it does not separate the problems the small model gets wrong. Bandit variants: sliding-window LinUCB equals LinUCB on this stationary stream, while linear Thompson sampling and ε-greedy explore more and lose 0.015 and 0.028 mean reward.

D. Headroom and Learning Dynamics

How much could any router gain on this data? The per-query oracle, which picks the model with the highest logged reward for each test problem, reaches a cumulative reward of 184.9, only 8.9 (5.1%) above small-only routing. It does so by sending 195 problems to Qwen3-4B, 13 to Qwen3-8B, and 2 to Qwen3-14B, which yields 99.5% accuracy at a cost of 1.54 per request. A cost-first oracle that calls the cheapest model that answers correctly reaches the same 99.5% at 1.51 per request, using Qwen3-4B for 200 problems, Qwen3-8B for 9, and Qwen3-14B for 1. The gap between 94.8% and 99.5% accuracy is therefore real and cheap to close in

hindsight, but closing it requires knowing in advance which 10 problems the small model will miss.

The warm-up phase shows how LinUCB arrives at its choice. With $\alpha = 1.0$, the router spent the 90 development queries on Qwen3-4B 49 times, Qwen3-14B 27 times, and Qwen3-8B 14 times, while the mean development rewards of the three models were 0.748, 0.692, and 0.644 for the 4B, 8B, and 14B models. Averaged over the test queries, the predicted reward is 0.758 for Qwen3-4B against 0.605 and 0.633 for the larger models, and the exploration bonuses are 0.166, 0.284, and 0.213. The bonus of the less-tried models narrows but does not close the gap in estimated reward, so the 4B model wins almost every comparison, and the remaining 1.8% of larger-model calls are the bonus occasionally tipping a close decision.

VII. System Implementation

The router is implemented in Python as a small package with four parts. The router module holds the feature extractor, the LinUCB policy with its per-model statistics, the four baselines, and the reward calculator. The evaluation module collects outcomes by sending every query to every model through vLLM's OpenAI-compatible endpoint; collection is resumable, so an interrupted run continues from the last logged pair. It also provides scorers for exact, multiple-choice, numeric, substring, unit-test, and judge-based quality, and the replay engine that produces Tables I-III. The experiments module runs the sweeps and the three bandit variants, and a unit-test suite covers the scorers, features, LinUCB updates, replay, and service.

For live use the same policy runs inside a FastAPI service. A /route request extracts features, scores the three models, calls the chosen one, and returns the answer with a decision identifier; a /feedback request attaches a quality rating and updates the router; and /stats reports its state. Every decision is written to an audit log, and a small web page lets a user ask a question, see which model was chosen, and rate the answer. A demo mode runs the routing and learning without a GPU.

VIII. Discussion and Limitations

The pilot supports two conclusions. First, the reward definition, not the algorithm, decides the outcome: when larger models add one or two correct answers per 210 problems at two to three times the cost, any reasonable cost weight makes the small model the right choice, and a learned router can at best match it. Second, LinUCB found that choice from 90 warm-up queries without hand-written rules, and its cost of exploration was small (0.002 reward per query). The situations in which per-query routing should pay off, namely several task categories with different best models and features that predict when the small model fails, are absent from this pilot by construction.

The size of the required accuracy gain makes this concrete. Because quality enters the reward with weight 1, the mean reward of a model equals its accuracy minus its mean cost and latency penalty. On the held-out split that penalty is 0.110 for Qwen3-4B, 0.216 for Qwen3-8B, and 0.286 for Qwen3-14B. A larger model therefore breaks even with the small one only if it is more accurate by the difference: 10.7 percentage points for Qwen3-8B and 17.7 points for Qwen3-14B. The measured gains are 0.9 and 0.4 points. On harder categories such as multi-step coding, where small models fail far more often, gaps of this size are plausible, which is why the full evaluation is needed before judging the method.

The limitations follow directly. The pilot covers one dataset, one task category, and 210 test queries, so accuracy differences of under one point are within noise and the per-category analysis (E3), workload shift (E5), and the effect of the category features cannot yet be assessed. Cost is a token-based proxy rather than measured GPU time, and latency was measured with one model served at a time on one GPU, without concurrent load. Results were obtained with FP8 checkpoints and thinking mode disabled and may not transfer to bf16, reasoning mode, or other model families. The reward weights were fixed for the pilot and not tuned. Standard LinUCB assumes stationary rewards, so drifting workloads may need sliding-window or discounted variants [16], and judge-based rewards for open-ended categories must be validated before use [20].

For deployment, the pilot suggests a simple procedure. Before routing at all, run each candidate on a small development sample and compute the reward of every fixed policy; if one model dominates under the chosen weights, as Qwen3-4B does here, a fixed or rule-based choice is sufficient and a learned router mainly acts as a safeguard that would notice if that stopped being true. LinUCB is worth its added state when the development data show different best models for different kinds of queries, and in that case the exploration coefficient should be kept small ($\alpha \leq 1$ in this pilot), because larger values spent up to 37% more cost for at most half a point of accuracy. Because the router keeps only a small summary for each model, its overhead is negligible next to a single LLM call, and its audit log makes every choice explainable.

IX. Conclusion and Future Work

This paper presented an adaptive LLM routing framework that formulates per-query model selection as a contextual-bandit problem and uses LinUCB to balance quality, latency, and cost without modifying the candidate models. In a GSM8K pilot with Qwen3-4B, 8B, and 14B on one RTX 4090, LinUCB learned to route 98.2% of queries to the smallest model, matched its 94.8% accuracy, and cut cost per request by 69% relative to large-only routing, but it did not exceed the small-only and rule-based baselines, which are near-optimal on a single-category workload.

Future work follows from these limits. The next step is the full seven-category evaluation, which adds MMLU-style question answering, HumanEval-style coding with unit tests, and judge-scored open-ended tasks, and which makes the per-category analysis (E3) and a workload-shift run (E5) possible. Further steps are to add features that predict when the small model will fail, to measure GPU-time cost under concurrent load, to compare budget-constrained routing [11], and to evaluate non-stationary and neural bandit variants.

REFERENCES

[1] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023.

[2] D. Ding et al., "Hybrid LLM: Cost-efficient and quality-aware query routing," in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[3] I. Ong et al., "RouteLLM: Learning to route LLMs from preference data," in Proc. Int. Conf. Learning Representations (ICLR), 2025.

[4] T. Shnitzer et al., "Large language model routing with benchmark datasets," arXiv:2309.15789, 2023.

[5] K. Lu et al., "Routing to the expert: Efficient reward-guided ensemble of large language models," in Proc. Conf. North American Chapter of the Association for Computational Linguistics (NAACL), 2024.

[6] Q. J. Hu et al., "RouterBench: A benchmark for multi-LLM routing system," arXiv:2403.12031, 2024.

[7] T. Lattimore and C. Szepesvári, Bandit Algorithms. Cambridge, U.K.: Cambridge Univ. Press, 2020.

[8] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in Proc. 19th Int. Conf. World Wide Web (WWW), 2010, pp. 661-670, doi: 10.1145/1772690.1772758.

[9] W. Chu, L. Li, L. Reyzin, and R. E. Schapire, "Contextual bandits with linear payoff functions," in Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS), PMLR vol. 15, 2011, pp. 208-214.

[10] X. Wang et al., "MixLLM: Dynamic routing in mixed large language models," in Proc. 2025 Conf. Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL), vol. 1, 2025, pp. 10912-10922, doi: 10.18653/v1/2025.naacl-long.545.

[11] P. Panda, R. Magazine, C. Devaguptapu, S. Takemori, and V. Sharma, "Adaptive LLM routing under budget constraints," in Findings of the Association for Computational Linguistics: EMNLP 2025, 2025, pp. 23934-23949.

[12] D. Jiang, X. Ren, and B. Y. Lin, "LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion," in Proc. 61st Annu. Meeting of the Association for Computational Linguistics (ACL), 2023.

[13] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," Machine Learning, vol. 47, no. 2-3, pp. 235-256, 2002.

[14] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári, "Improved algorithms for linear stochastic bandits," in Advances in Neural Information Processing Systems (NeurIPS), vol. 24, 2011.

[15] L. Li, W. Chu, J. Langford, and X. Wang, "Unbiased offline evaluation of contextual-bandit-based news article recommendation algorithms," in Proc. 4th ACM Int. Conf. Web Search and Data Mining (WSDM), 2011.

[16] A. Garivier and E. Moulines, "On upper-confidence bound policies for switching bandit problems," in Proc. 22nd Int. Conf. Algorithmic Learning Theory (ALT), LNCS vol. 6925, 2011, pp. 174-188.

[17] D. Hendrycks et al., "Measuring massive multitask language understanding," in Proc. Int. Conf. Learning Representations (ICLR), 2021.

[18] K. Cobbe et al., "Training verifiers to solve math word problems," arXiv:2110.14168, 2021.

[19] M. Chen et al., "Evaluating large language models trained on code," arXiv:2107.03374, 2021.

[20] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, vol. 36, 2023.

[21] A. Yang et al., "Qwen3 technical report," arXiv:2505.09388, 2025.

[22] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in Proc. 29th ACM Symp. Operating Systems Principles (SOSP), 2023, doi: 10.1145/3600006.3613165.

[23] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. Conf. Empirical Methods in Natural Language Processing and Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 3982-3992.

[24] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. New York, NY, USA: Chapman & Hall, 1993.